Quantum-Resistant Cryptography via Lattice-Based Systems

Complete Project Plan, Theory, and Work Distribution

Atharva Sawant (2024ADPS0872G)
Prateek Singhal (2024ADPS0830G)
Shamit

September 27, 2026

Abstract

This document consolidates the complete plan, theory, and work distribution for a from-scratch Ring-LWE cryptographic prototype with empirical security analysis. It includes the original project proposal, a 15-day alternative, a full 1-month 3-member plan, a complete theory guide, a comparison framework against RSA and ECC, and a fair distribution of responsibilities among three team members. The final deliverable is a reproducible GitHub repository containing a Ring-LWE encryption implementation with NTT, a from-scratch LLL lattice attack, parameter-sweep experiments, plots, a written report, and a live demo.

Contents

1 Original Project Proposal (from PPT) 4
1.1 Motivation . . . 4
1.2 Methodology . . . 4
1.3 Possible Outcomes . . . 5

2 15-Day Alternative Plan 5
2.1 Recommended Alternative: From-Scratch Toy LWE + Parameter/Attack Study . 5
2.2 The LWE Scheme You Can Implement (Regev-Style, Simplified) . . . 5
2.3 15-Day Plan . . . 6
2.4 Optional Variations . . . 6
2.5 How to Prove It's Your Own Work . . . 6

3 1-Month, 3-Member Ring-LWE Project 7
3.1 Overview . . . 7
3.2 Project Title . . . 7
3.3 Why Ring-LWE Instead of Basic LWE . . . 7
3.4 Team Roles . . . 7
3.5 The Cryptosystem You Will Build . . . 7
3.6 30-Day Plan . . . 8
3.6.1 Week 1: Foundations and Polynomial Arithmetic (Days 1–7) . . . 8
3.6.2 Week 2: NTT Optimization and Correctness (Days 8–14) . . . 8
3.6.3 Week 3: Experiments and Parameter Sweep (Days 15–21) . . . 9
3.6.4 Week 4: Write-up and Viva Preparation (Days 22–30) . . . 9
3.7 What Makes This Original (Not a Downloaded Simulation) . . . 9
3.8 Deliverables Summary . . . 10

3.9 Key References to Read in Week 1 . . . 10
3.10 How to Prove Ownership During Viva . . . 10

4 Final Product Description **10**
4.1 What You Actually Submit . . . 11
4.1.1 1. Code Repository . . . 11
4.1.2 2. Working Ring-LWE Prototype . . . 11
4.1.3 3. From-Scratch LLL Attack . . . 11
4.1.4 4. Experimental Outputs . . . 11
4.1.5 5. Written Report . . . 12
4.1.6 6. Live Demo . . . 12
4.2 What the Final Product Is NOT . . . 12
4.3 The Single Most Important Final Artifact . . . 12

5 Comparison with RSA and ECC **13**
5.1 Why This Comparison Is Powerful . . . 13
5.2 Comparison Framework: What to Measure . . . 13
5.3 Expected Results and Data Points (128-bit Security) . . . 13
5.4 How to Integrate This into Your Project . . . 13
5.5 How This Addresses "Originality" . . . 14

6 Complete Theory Guide **14**
6.1 Part 1: Why Quantum Computers Break RSA and ECC . . . 14
6.1.1 1.1 Classical cryptography's foundation . . . 14
6.1.2 1.2 Shor's algorithm . . . 14
6.1.3 1.3 Why lattice problems are different . . . 14
6.2 Part 2: Lattices — The Geometric Foundation . . . 14
6.2.1 2.1 What is a lattice? . . . 14
6.2.2 2.2 The Shortest Vector Problem (SVP) . . . 15
6.2.3 2.3 The Closest Vector Problem (CVP) . . . 15
6.2.4 2.4 Why these problems are hard . . . 15
6.2.5 2.5 The connection to LWE . . . 15
6.3 Part 3: Learning With Errors (LWE) . . . 15
6.3.1 3.1 The basic setup . . . 15
6.3.2 3.2 The search problem . . . 15
6.3.3 3.3 The decision problem . . . 15
6.3.4 3.4 Why is LWE hard? . . . 15
6.3.5 3.5 LWE encryption (Regev's scheme) . . . 15
6.4 Part 4: Ring-LWE — Making LWE Practical . . . 16
6.4.1 4.1 The problem with basic LWE . . . 16
6.4.2 4.2 The ring structure . . . 16
6.4.3 4.3 Ring-LWE relation . . . 16
6.4.4 4.4 Ring-LWE encryption . . . 16
6.4.5 4.5 Why Ring-LWE is secure . . . 16
6.5 Part 5: Polynomial Arithmetic in $\mathbb{Z}_q[X]/(X^n+1)$ . . . 17
6.5.1 5.1 Representation . . . 17
6.5.2 5.2 Addition . . . 17
6.5.3 5.3 Multiplication (naive) . . . 17
6.5.4 5.4 Multiplication (NTT) . . . 17
6.5.5 5.5 Why X^n+1? . . . 17
6.6 Part 6: The LLL Algorithm . . . 17
6.6.1 6.1 Purpose . . . 17

6.6.2 6.2 Gram-Schmidt orthogonalization . . . 17
6.6.3 6.3 LLL conditions . . . 17
6.6.4 6.4 The algorithm . . . 17
6.6.5 6.5 LLL for LWE attack . . . 18
6.7 Part 7: Comparison with RSA and ECC . . . 18
6.7.1 7.1 Security foundation . . . 18
6.7.2 7.2 Key and ciphertext sizes (128-bit security) . . . 18
6.7.3 7.3 Performance . . . 18
6.7.4 7.4 The trade-off . . . 18
6.8 Part 8: Summary — What You Need to Know for Each Part . . . 18
6.9 Suggested Reading Order . . . 19
6.10 Key Equations to Memorize . . . 19
6.11 What to Do Next . . . 19

7 Work Distribution Among 3 Members 19
7.1 Role Assignment Overview . . . 20
7.2 Weekly Work Distribution . . . 20
7.2.1 Week 1 — Foundations & Naive Implementation . . . 20
7.2.2 Week 2 — NTT & Attack Implementation . . . 21
7.2.3 Week 3 — Experiments & Data Collection . . . 21
7.2.4 Week 4 — Integration, Report, Viva Prep . . . 21
7.3 Cross-Training Schedule (Critical) . . . 22
7.4 Handoff & Dependency Schedule . . . 22
7.5 Shared Responsibilities . . . 22
7.6 Fairness Check . . . 23
7.7 Final Product Ownership . . . 23

1 Original Project Proposal (from PPT)

Title

Quantum-Resistant Cryptography via Lattice-Based Systems
Scoping a simplified Learning-With-Errors (LWE) encryption prototype for a 3-month timeline
Authors: Atharva Sawant (2024ADPS0872G), Prateek Singhal (2024ADPS0830G), Shamit

1.1 Motivation

- **Quantum threat to classical cryptography.** A sufficiently large quantum computer running Shor's algorithm can efficiently break RSA and ECC — the same number-theoretic assumptions this course builds toward.
- **Lattice problems resist quantum attack.** The Shortest and Closest Vector Problems (SVP/CVP) have no known efficient quantum algorithm, making lattice-based schemes leading post-quantum candidates.
- **Validated by real-world standards.** NIST's finalized post-quantum standards, Kyber and Dilithium, are both lattice-based — this topic is directly aligned with where the industry is heading.
- **A direct replacement for RSA/ECC.** LWE is a public-key primitive built on modular linear algebra ($A \cdot s + e \bmod q$) — a quantum-safe swap-in for the asymmetric role RSA and ECC play today.

1.2 Methodology

A self-contained 3-month roadmap, independent of the classical-cipher coursework track. A single, self-contained primitive — not a full lattice library — kept to one clean deliverable per month.

Month 1: Foundations

- Modular arithmetic & vector-space foundations for LWE.
- Study the core LWE relation: $A \cdot s + e \pmod q$.
- Parameter scoping — how n, q, σ trade off security vs. correct decryption.

Month 2: Core Build

- Build toy-scale LWE key generation, encryption & decryption in Python.
- Package it all as a single, runnable Jupyter notebook.
- Sanity-check correct decryption at the chosen toy parameters.

Month 3: Analysis

- Plot empirical noise growth against the $q/4$ decryption-failure threshold.
- Compile a theoretical key/ciphertext-size table vs RSA-2048/4096 & ECC-256.
- Write up findings in the notebook + a short report.

1.3 Possible Outcomes

Three deliverables, scoped to what a 3-month student timeline can realistically finish.

1. **Python LWE prototype.** A clean, runnable demo of toy-scale key generation, encryption, and decryption — packaged as a single Jupyter notebook.
2. **Noise threshold graph.** An empirical curve plotting error growth against the $q/4$ point where decryption starts to fail.
3. **Key-size comparison.** A theoretical table comparing LWE key/ciphertext sizes against RSA-2048/4096 and ECC-256.

2 15-Day Alternative Plan

Context

The professor's comment means: *don't just download a demo or paste AI-generated code.* They want evidence that *you* understand the math, can build it step by step, and can investigate something original.

For a 15-day, no-prior-knowledge version, drop the full 3-month scope and do a much smaller, from-scratch, experimental LWE project. The goal is not a production library. The goal is: **build a toy LWE cryptosystem yourself, then use it to study parameter trade-offs, decryption failure, and security scaling.**

2.1 Recommended Alternative: From-Scratch Toy LWE + Parameter/Attack Study

Deliverables

1. A Jupyter notebook that implements LWE **without external crypto/lattice libraries** (pure Python + `random`).
2. A hand-worked example for tiny parameters ($n = 2$, $q = 97$) showing every number.
3. A parameter-sweep experiment: vary n, q, noise σ, and measure decryption failure.
4. A noise-threshold graph: error growth vs $q/4$ failure boundary.
5. A brute-force attack on tiny secrets to show how security scales with n.
6. A key/ciphertext size table comparing LWE to RSA-2048/4096 and ECC-256.
7. A short report explaining why LWE is post-quantum and what your experiments show.

2.2 The LWE Scheme You Can Implement (Regev-Style, Simplified)

Use only modular arithmetic.

- Parameters: n (secret dimension), q (modulus), m (samples), σ (noise).
- Secret: $s \in \mathbb{Z}_q^n$, often binary or small.
- Public matrix: $A \in \mathbb{Z}_q^{m \times n}$.
- Error: $e \in \mathbb{Z}_q^m$, small.
- Public key: $(A,\ b = As + e \bmod q)$.

Encrypt one bit $\mu \in \{0, 1\}$:

- Pick random $x \in \{0,1\}^m$.
- $u = A^T x \bmod q$.
- $v = b^T x + \mu \cdot \lfloor q/2 \rfloor \bmod q$.
- Ciphertext $= (u, v)$.

Decrypt:

- $d = v - s^T u \bmod q$.
- If d is close to $0 \rightarrow$ plaintext 0.
- If d is close to $q/2 \rightarrow$ plaintext 1.
- Correct decryption holds when the accumulated noise is $< q/4$.

That $q/4$ threshold is exactly what your noise graph will show.

2.3 15-Day Plan

Days	Task
1–2	Math foundations: modular arithmetic, vectors, matrices over $\mathbb{Z}_q$. Hand-compute tiny example.
3–4	Pure Python building blocks: `mod_add`, `mod_mul`, matrix-vector multiply, transpose. No NumPy at first.
5–6	LWE key generation + brute-force attack. Plot time vs n.
7–8	Encryption/decryption. Test bits 0 and 1, count failures.
9–10	Parameter sweep + noise graph. Vary n, q, σ, m. Plot empirical noise vs $q/4$.
11–12	Size comparison: LWE key/ciphertext sizes vs RSA-2048/4096, ECC-256.
13–14	Notebook + report. Write like a lab report.
15	Viva preparation.

2.4 Optional Variations

- **Spreadsheet-first LWE** — do $n = 2$, $q = 97$ in Excel.
- **Only LWE search problem** — skip encryption, focus on recovering s.
- **LWE vs RSA toy comparison** — implement both from scratch.

2.5 How to Prove It's Your Own Work

- Keep a dated lab notebook.
- Write code in small Git commits.
- Avoid copy-pasting large AI blocks.
- Include a manual walkthrough for $n = 2$, $q = 97$.
- Be ready to modify code live during viva.

3 1-Month, 3-Member Ring-LWE Project

3.1 Overview

This builds on the 15-day plan with a much more ambitious scope. With one month and three people, you can move beyond a toy Regev-style scheme and implement a **Ring-LWE cryptosystem from scratch**, then conduct an **empirical security analysis** against lattice reduction attacks.

This addresses the professor's concern directly: you're not downloading a simulation, you're building the polynomial arithmetic, the number-theoretic transform, and the attack reduction yourself.

3.2 Project Title

From-Scratch Ring-LWE: Implementation, Optimization, and Empirical Security Analysis

3.3 Why Ring-LWE Instead of Basic LWE

Basic LWE uses large matrices, making keys and ciphertexts enormous. Ring-LWE replaces matrices with polynomials in the ring $\mathbb{Z}_q[X]/(X^n + 1)$, where n is a power of two. This is the foundation of **NewHope**, the key-exchange protocol deployed by Google in Chrome, and **Kyber**, NIST's post-quantum KEM standard.

Ring-LWE is significantly more complex than basic LWE: you must implement polynomial arithmetic mod $X^n + 1$, modular reduction, noise sampling from discrete Gaussian distributions, and ideally the Number-Theoretic Transform for efficient multiplication. That complexity is exactly what makes it appropriate for a 1-month, 3-person project.

3.4 Team Roles

Role	Owner	Responsibility
A: Polynomial Arithmetic	Member 1	Implement $\mathbb{Z}_q[X]/(X^n + 1)$ arithmetic, modular reduction, noise sampling
B: LWE Protocol & NTT	Member 2	Build keygen/encrypt/decrypt; implement and integrate the Number-Theoretic Transform
C: Security Analysis	Member 3	Implement LLL lattice reduction, construct the LWE attack lattice, run parameter sweeps

All three write their own sections of the final report and notebook.

3.5 The Cryptosystem You Will Build

Ring-LWE encryption (simplified NewHope-style):

- **Parameters:** $n = 256$ (polynomial degree), $q = 12289$ (prime, $q \equiv 1 \bmod 2n$), $\sigma \approx 3.2$ (noise standard deviation).
- **Secret key:** $s \in R_q$, sampled from a small distribution (e.g., binary or discrete Gaussian).
- **Public key:** $(a,\ b = a \cdot s + e \bmod q)$ where a is uniformly random in R_q and e is small noise.

- **Encrypt bit μ:** sample r, e_1, e_2; compute $u = a \cdot r + e_1$, $v = b \cdot r + e_2 + \mu \cdot \lfloor q/2 \rfloor$.
- **Decrypt:** $d = v - s \cdot u \bmod q$; output 0 if d is close to 0, 1 if close to $q/2$.

The critical difference from basic LWE: all operations are **polynomial multiplications in the ring**, not matrix-vector products.

3.6 30-Day Plan

3.6.1 Week 1: Foundations and Polynomial Arithmetic (Days 1–7)

Goal: Implement the ring arithmetic layer from scratch, no external crypto libraries.

Day	Member 1	Member 2	Member 3
1–2	Learn modular arithmetic, polynomial representation $\sum a_i X^i \bmod X^n + 1$	Learn Ring-LWE definition, read Regev 2005 and Lyubashevsky et al. 2010	Learn lattice basics: what is a lattice, SVP, CVP
3–4	Implement `poly_add`, `poly_sub`, `poly_mul_naive` (schoolbook), `poly_mod_reduction`	Hand-work a tiny example: $n = 4$, $q = 17$, encrypt/decrypt bit 0 and 1	Study LLL algorithm; implement Gram-Schmidt on paper
5–6	Implement discrete Gaussian sampler (rejection sampling from a table)	Implement key generation, encryption, decryption using naive multiplication	Write Python LLL implementation for small integer matrices
7	**Integration test:** encrypt/decrypt 100 bits, verify correctness	**Integration test:** same	**Integration test:** verify LLL on a 2×2 example

Deliverable: A working, slow Ring-LWE implementation with naive polynomial multiplication.

3.6.2 Week 2: NTT Optimization and Correctness (Days 8–14)

Goal: Replace naive $O(n^2)$ multiplication with $O(n \log n)$ NTT, verify correctness.

Day	Member 1	Member 2	Member 3
8–9	Learn NTT over $\mathbb{Z}_q$; find primitive $2n$-th root of unity mod q	Implement NTT-based multiplication; test against naive on random polynomials	Study how to embed LWE into a lattice for attack
10–11	Implement NTT butterfly and inverse NTT	Integrate NTT into keygen/encrypt/decrypt; verify output matches naive version	Construct the attack lattice: $\Lambda = \{(A, b) : b = As + e\}$ with the "q-ary" embedding
12–13	Benchmark: time naive vs NTT for $n = 64, 128, 256$	Run 1000 encrypt/decrypt cycles; measure failure rate	Implement LLL-based attack for $n = 8, 16, 32$; measure time to recover s

Day	Member 1	Member 2	Member 3
14	**Integration test:** NTT version produces identical ciphertexts to naive	**Integration test:** same	**Integration test:** LLL recovers secret for tiny n

Deliverable: A fast Ring-LWE implementation with NTT, plus a working (slow) LLL attack for small parameters.

3.6.3 Week 3: Experiments and Parameter Sweep (Days 15–21)

Goal: Run systematic experiments, generate the data for your report.

Day	Member 1	Member 2	Member 3
15–16	Sweep $n = 64, 128, 256$; measure key/ciphertext sizes	Sweep $\sigma = 1, 2, 3.2, 5$; measure decryption failure rate	Sweep $n = 8, 12, 16, 20$; measure LLL attack time
17–18	Plot key size vs n; compare to theoretical formula $2n \log_2 q$ bits	Plot failure rate vs σ; identify the $q/4$ noise threshold	Plot attack time vs n; fit exponential curve
19–20	Compare sizes: Ring-LWE ($n = 256$) vs RSA-2048/4096 vs ECC-256	Verify: does failure rate match theoretical bound?	Test: can LLL break $n = 24$? Document where it fails
21	**Checkpoint:** all plots generated, data saved as CSV	**Checkpoint:** same	**Checkpoint:** same

Deliverable: Five to six publication-quality plots: size comparison, failure rate, attack scaling, NTT speedup.

3.6.4 Week 4: Write-up and Viva Preparation (Days 22–30)

Day	All Members
22–23	Write the notebook as a lab report: Introduction → Mathematical Background → Implementation → Experiments → Results → Discussion
24–25	Write the standalone report (8–10 pages); each member writes their own section
26–27	Prepare the live demo: show keygen → encrypt → decrypt → LLL attack on tiny parameters
28–29	Rehearse viva answers: be ready to modify code live, explain every line
30	Final submission and presentation

3.7 What Makes This Original (Not a Downloaded Simulation)

1. **You implement NTT yourself.** Finding the primitive root of unity, building the butterfly, handling the bit-reversal permutation — this is real cryptographic engineering.
2. **You implement LLL yourself.** Not calling `fpylll`, but writing the Gram-Schmidt, size reduction, and swap steps.

3. **You run your own parameter sweep.** The plots and tables are *your* results, not copied from a paper.
4. **You document where your implementation fails.** For example: "LLL recovers the secret for $n \leq 20$ in under 10 seconds, but times out for $n = 24$." That's a genuine empirical finding.

3.8 Deliverables Summary

Deliverable	Description
Jupyter Notebook	Complete from-scratch Ring-LWE implementation with NTT, plus LLL attack
Report (8–10 pages)	Mathematical background, implementation details, experimental results, discussion
Noise/Attack Plot	Log-scale plot of LLL attack time vs n, with exponential fit
Size Comparison Table	Ring-LWE vs RSA-2048/4096 vs ECC-256 (key size, ciphertext size)
Live Demo	Keygen $\rightarrow$ encrypt $\rightarrow$ decrypt $\rightarrow$ attack in under 5 minutes

3.9 Key References to Read in Week 1

- Regev, *On Lattices, Learning with Errors, Random Linear Codes, and Cryptography* (2005).
- Lyubashevsky, Peikert, Regev, *On Ideal Lattices and Learning with Errors over Rings* (2010).
- The NewHope specification (Alkim, Ducas, Pöppelmann, Schwabe).
- Lenstra, Lenstra, Lovász, *Factoring Polynomials with Rational Coefficients* (1982).

3.10 How to Prove Ownership During Viva

- Keep a **dated lab notebook**.
- Make **Git commits** with meaningful messages.
- Be ready to **break your own code live**.
- Explain **why** NTT works: why $q \equiv 1 \bmod 2n$ is required, why the ring is $X^n + 1$ and not $X^n - 1$.

4 Final Product Description

The final product is **not a production-ready encryption library**. It is a **from-scratch Ring-LWE cryptographic prototype plus an empirical security study**, packaged so your professor can run it, inspect it, and question it live.

In one sentence:

> **A reproducible GitHub repository containing a from-scratch Ring-LWE encryption implementation with NTT, a from-scratch LLL lattice attack, parameter-sweep experiments, plots, a written report, and a live demo.**

4.1 What You Actually Submit

4.1.1 1. Code Repository

A clean Python project, no external crypto/lattice libraries.

```
ringlwe-project/
|-- ringlwe/
| |-- poly.py # Polynomial arithmetic mod X^n + 1
| |-- ntt.py # NTT / inverse NTT, primitive root search
| |-- sampler.py # Discrete Gaussian noise sampler
| |-- rlwe.py # Keygen, encrypt, decrypt
| |-- lll.py # From-scratch LLL lattice reduction
| `-- experiments.py # Parameter sweeps, failure-rate tests
|-- notebooks/
| |-- 01_hand_example.ipynb
| |-- 02_ringlwe_demo.ipynb
| |-- 03_ntt_speedup.ipynb
| |-- 04_lll_attack.ipynb
| `-- 05_results.ipynb
|-- data/ # CSV outputs from experiments
|-- plots/ # Generated graphs
|-- report.pdf # 8-10 page report
|-- slides.pdf # Presentation
`-- README.md # How to run everything
```

4.1.2 2. Working Ring-LWE Prototype

- Generate Ring-LWE keys with $n = 64, 128, 256$, $q = 12289$, noise $\sigma \approx 3.2$.
- Encrypt and decrypt individual bits.
- Use naive polynomial multiplication and NTT-based multiplication.
- Show that NTT is much faster for larger n.
- Measure decryption failure rate as noise grows.
- Show where noise crosses the $q/4$ threshold and decryption breaks.

4.1.3 3. From-Scratch LLL Attack

- A working LLL implementation written by your team.
- An attack lattice for small Ring-LWE instances.
- Empirical results showing LLL recovers the secret for $n \leq 20$ or so, and fails/times out for larger n.
- A plot of attack time vs n, fitted to an exponential curve.

4.1.4 4. Experimental Outputs

- **NTT speedup:** naive vs NTT multiplication time for $n = 64, 128, 256$.
- **Noise threshold graph:** decryption failure rate vs noise σ, with the $q/4$ boundary marked.
- **Attack scaling graph:** LLL attack time vs secret dimension n.

- **Size comparison table:** Ring-LWE key/ciphertext sizes vs RSA-2048, RSA-4096, ECC-256.
- **Correctness data:** 1000+ encrypt/decrypt trials at chosen parameters.

4.1.5 5. Written Report

An 8–10 page PDF with:

- Introduction: quantum threat, why lattices.
- Mathematical background: rings, $\mathbb{Z}_q[X]/(X^n + 1)$, LWE, Ring-LWE.
- Implementation details: NTT, sampling, keygen, encrypt, decrypt.
- Attack methodology: LLL and the embedding lattice.
- Results: all plots and tables.
- Discussion: limitations, what would be needed for real security.
- Individual contributions from each of the three members.

4.1.6 6. Live Demo

A 5-minute demo where you can:

1. Generate keys.
2. Encrypt bit 0 and bit 1.
3. Decrypt correctly.
4. Increase noise until decryption fails.
5. Run LLL on a tiny instance and recover the secret.
6. Change n or q live and explain what happens.

4.2 What the Final Product Is NOT

- It is **not** Kyber, NewHope, or a secure real-world system.
- It is **not** constant-time or side-channel resistant.
- It is **not** CCA-secure.
- It uses **toy parameters** for the attack experiments.
- LLL only breaks very small dimensions; real Ring-LWE uses much larger n and stronger attacks.

4.3 The Single Most Important Final Artifact

If your professor asks, "Show me the final product," you point to:

The Jupyter notebook + repository + report.

The notebook should be self-contained: it explains the math, shows the code, runs the experiments, and displays the plots. The repository proves it is reproducible. The report proves you understand it. The live demo proves you can defend it.

That is the final product: **a from-scratch Ring-LWE prototype and empirical lattice-security study, not a downloaded simulation.**

5 Comparison with RSA and ECC

Yes, absolutely. Comparing your Ring-LWE prototype against RSA and ECC is a natural and highly valuable extension of your project. It provides the critical context that justifies the entire field of post-quantum cryptography.

5.1 Why This Comparison Is Powerful

The central narrative is a trade-off:

- **RSA & ECC**: Efficient with small keys, but **completely broken** by a sufficiently large quantum computer running Shor's algorithm.
- **Ring-LWE**: Less efficient with larger keys, but **believed to be resistant** to quantum attacks.

5.2 Comparison Framework: What to Measure

1. **Key and Ciphertext Sizes**
2. **Computational Performance**
3. **Security Level**

5.3 Expected Results and Data Points (128-bit Security)

Metric	**RSA-3072**	**ECDH P-256**	**Ring-LWE (NewHope)**
Quantum Security	Broken by Shor's	Broken by Shor's	Resistant
Public Key Size	384 Bytes	64 Bytes	∼1,824 Bytes
Ciphertext Size	384 Bytes	64 Bytes	∼2,048 Bytes
Key Generation Speed	Slow (ms)	Very Fast (μs)	Fast (μs)
Encryption/Signing Speed	Slow (ms)	Fast (μs)	Very Fast (μs)

Key Insight:

- Ring-LWE public keys are roughly **4–5× larger** than RSA-3072 and about **30× larger** than ECDH P-256.
- However, in terms of speed, optimized lattice-based schemes like Ring-LWE are often **faster than RSA** and **comparable to or faster than ECC**.

5.4 How to Integrate This into Your Project

- **Week 3 (Days 15–21)** dedicated to comparison.
- Use Python's `cryptography` or `PyCryptodome` for RSA/ECC baselines (acceptable since your from-scratch work is on Ring-LWE).
- Measure your Ring-LWE prototype: serialize keys/ciphertexts, benchmark operations.

- Build size comparison table and performance plots.
- Connect to security: highlight the trade-off.

5.5 How This Addresses "Originality"

- **You are building a benchmarking framework**, not just a single piece of software.
- **You are generating your own data** to validate claims from papers.
- **You are demonstrating deep understanding** of the "why" behind post-quantum cryptography.

This transforms your project from a pure implementation exercise into a more complete **research and analysis project**.

6 Complete Theory Guide

This is everything you need to understand before you write a single line of code. Read it in order. Each section builds on the previous one.

6.1 Part 1: Why Quantum Computers Break RSA and ECC

6.1.1 1.1 Classical cryptography's foundation

RSA relies on factoring a large number $N = p \cdot q$. ECC relies on the Elliptic Curve Discrete Logarithm Problem (ECDLP). Both are hard for classical computers.

6.1.2 1.2 Shor's algorithm

In 1994, Peter Shor showed that a **quantum computer** can solve both factoring and discrete logarithm in **polynomial time**.

- **Classical**: Best factoring algorithm takes sub-exponential time.
- **Quantum (Shor)**: Takes roughly $O(n^3)$ time.

This means RSA-2048 could be broken in hours or days on a sufficiently large quantum computer.

6.1.3 1.3 Why lattice problems are different

Lattice problems (SVP, CVP) have **no known efficient quantum algorithm**. Shor's algorithm does not apply because factoring and discrete log have hidden algebraic structure that quantum Fourier sampling exploits. Lattice problems do not appear to have this structure.

6.2 Part 2: Lattices — The Geometric Foundation

6.2.1 2.1 What is a lattice?

A **lattice** is a set of points in n-dimensional space formed by integer combinations of linearly independent vectors.

Formally: Given n linearly independent vectors $\mathbf{b}_1, \mathbf{b}_2, \ldots, \mathbf{b}_n \in \mathbb{R}^m$, the lattice is:

$$\mathcal{L} = \left\{ \sum_{i=1}^{n} z_i \mathbf{b}_i : z_i \in \mathbb{Z} \right\}$$

6.2.2 2.2 The Shortest Vector Problem (SVP)

Given a lattice basis, find the **shortest non-zero vector** in the lattice.

6.2.3 2.3 The Closest Vector Problem (CVP)

Given a lattice $\mathcal{L}$ and a target point $\mathbf{t} \in \mathbb{R}^m$, find the lattice point **closest** to $\mathbf{t}$.

6.2.4 2.4 Why these problems are hard

Best known algorithms run in exponential time $2^{O(n)}$ for exact solutions. No quantum speedup is known that would make these easy.

6.2.5 2.5 The connection to LWE

LWE is an **average-case** problem that reduces to **worst-case** lattice problems.

6.3 Part 3: Learning With Errors (LWE)

6.3.1 3.1 The basic setup

Parameters: n (dimension), q (modulus), χ (error distribution).

The LWE relation:

$$\mathbf{b} = \mathbf{A} \cdot \mathbf{s} + \mathbf{e} \pmod{q}$$

6.3.2 3.2 The search problem

Given $\mathbf{A}$ and $\mathbf{b}$, find $\mathbf{s}$. With error, it's hard.

6.3.3 3.3 The decision problem

Distinguish LWE distribution from uniform random.

6.3.4 3.4 Why is LWE hard?

The error "smears" the linear structure. Finding $\mathbf{s}$ is like solving noisy linear equations, which reduces to finding a short vector in a lattice.

6.3.5 3.5 LWE encryption (Regev's scheme)

Key generation:

- Choose $\mathbf{s} \in \mathbb{Z}_q^n$.
- Choose $\mathbf{A} \in \mathbb{Z}_q^{m \times n}$ uniformly.
- Choose $\mathbf{e} \in \mathbb{Z}_q^m$ from χ.
- Compute $\mathbf{b} = \mathbf{A}\mathbf{s} + \mathbf{e} \mod q$.
- Public key: $(\mathbf{A}, \mathbf{b})$. Secret key: $\mathbf{s}$.

Encryption of bit $\mu \in \{0, 1\}$:

- Choose random $\mathbf{x} \in \{0, 1\}^m$.
- Compute $\mathbf{u} = \mathbf{A}^T\mathbf{x} \mod q$.
- Compute $v = \mathbf{b}^T\mathbf{x} + \mu \cdot \lfloor q/2 \rfloor \mod q$.

- Ciphertext: $(\mathbf{u}, v)$.

Decryption:

- Compute $d = v - \mathbf{s}^T\mathbf{u} \mod q$.
- If d is close to 0, output 0; if close to $q/2$, output 1.

Why it works: The noise term $\mathbf{e}^T\mathbf{x}$ is small. Decryption fails when $|\mathbf{e}^T\mathbf{x}| > q/4$.

6.4 Part 4: Ring-LWE — Making LWE Practical

6.4.1 4.1 The problem with basic LWE

Key and ciphertext sizes are huge.

6.4.2 4.2 The ring structure

The ring: $R_q = \mathbb{Z}_q[X]/(X^n + 1)$

Where n is a power of 2, q is a prime such that $q \equiv 1 \mod 2n$.

6.4.3 4.3 Ring-LWE relation

$$b = a \cdot s + e \in R_q$$

6.4.4 4.4 Ring-LWE encryption

Key generation:

- $s \leftarrow \chi^n$
- $a \leftarrow R_q$
- $e \leftarrow \chi^n$
- $b = a \cdot s + e$
- Public key: (a, b). Secret key: s.

Encryption of bit μ:

- $r, e_1, e_2 \leftarrow \chi^n$
- $u = a \cdot r + e_1$
- $v = b \cdot r + e_2 + \mu \cdot \lfloor q/2 \rfloor$
- Ciphertext: (u, v).

Decryption:

- $d = v - s \cdot u$
- Noise term: $e \cdot r - s \cdot e_1 + e_2$. If less than $q/4$, decryption succeeds.

6.4.5 4.5 Why Ring-LWE is secure

At least as hard as certain worst-case lattice problems in **ideal lattices**.

6.5 Part 5: Polynomial Arithmetic in $\mathbb{Z}_q[X]/(X^n+1)$

6.5.1 5.1 Representation

Coefficient vector: $a = (a_0, a_1, \ldots, a_{n-1})$.

6.5.2 5.2 Addition

Component-wise mod q.

6.5.3 5.3 Multiplication (naive)

Schoolbook, $O(n^2)$.

6.5.4 5.4 Multiplication (NTT)

Number-Theoretic Transform: evaluate at n points, multiply pointwise, interpolate back. Requires primitive $2n$-th root of unity ω in $\mathbb{Z}_q$.

NTT:

$$\hat{a}_i = \sum_{j=0}^{n-1} a_j \omega^{ij} \mod q$$

Complexity: $O(n \log n)$.

6.5.5 5.5 Why $X^n + 1$?

Irreducible over $\mathbb{Z}_q$ when $q \equiv 1 \mod 2n$ and n is a power of 2. NTT works cleanly.

6.6 Part 6: The LLL Algorithm

6.6.1 6.1 Purpose

Takes a "bad" lattice basis and produces a "good" basis with shorter, nearly orthogonal vectors.

6.6.2 6.2 Gram-Schmidt orthogonalization

$$\mathbf{b}_i^* = \mathbf{b}_i - \sum_{j=1}^{i-1} \mu_{i,j} \mathbf{b}_j^*$$

$$\mu_{i,j} = \frac{\langle \mathbf{b}_i, \mathbf{b}_j^* \rangle}{\langle \mathbf{b}_j^*, \mathbf{b}_j^* \rangle}$$

6.6.3 6.3 LLL conditions

1. **Size reduction:** $|\mu_{i,j}| \leq 1/2$
2. **Lovász condition:** $\|\mathbf{b}_i^*\|^2 \geq (\delta - \mu_{i,i-1}^2)\|\mathbf{b}_{i-1}^*\|^2$

6.6.4 6.4 The algorithm

```
Input: basis B = {b_1, ..., b_n}
Output: LLL-reduced basis

k = 1
while k < n:
```

```
    for j = k-1 down to 0:
        if |mu_{k,j}| > 1/2:
            b_k = b_k - round(mu_{k,j}) * b_j
    if ||b_k*||^2 >= (delta - mu_{k,k-1}^2) * ||b_{k-1}*||^2:
        k = k + 1
    else:
        swap(b_k, b_{k-1})
        k = max(k-1, 1)
```

6.6.5 6.5 LLL for LWE attack

Construct the q-ary lattice $\Lambda_q(\mathbf{A})$. LLL works for very small n (e.g., $n \leq 20$). For larger n, need BKZ, sieving.

6.7 Part 7: Comparison with RSA and ECC

6.7.1 7.1 Security foundation

Scheme	Hard Problem	Quantum Threat
RSA	Integer factorization	Broken by Shor
ECC	Elliptic curve discrete log	Broken by Shor
LWE/Ring-LWE	Worst-case lattice problems	No known efficient quantum algorithm

6.7.2 7.2 Key and ciphertext sizes (128-bit security)

Scheme	Public Key	Ciphertext/Signature
RSA-3072	384 bytes	384 bytes
ECDH P-256	64 bytes	64 bytes
Ring-LWE (NewHope)	∼1,824 bytes	∼2,048 bytes

6.7.3 7.3 Performance

Operation	RSA-3072	ECDH P-256	Ring-LWE
Key generation	Slow (ms)	Fast (μs)	Fast (μs)
Encryption	Slow (ms)	Fast (μs)	Very fast (μs)
Decryption	Slow (ms)	Fast (μs)	Very fast (μs)

6.7.4 7.4 The trade-off

- **RSA/ECC:** Small keys, fast operations, but **quantum-vulnerable**.
- **Ring-LWE:** Larger keys, competitive speed, but **quantum-resistant**.

6.8 Part 8: Summary — What You Need to Know for Each Part

Project Component	Theory Needed
Polynomial arithmetic	Part 5.1–5.3
NTT	Part 5.4–5.5
Noise sampling	Part 3.1
Key generation	Part 4.4
Encryption/decryption	Part 4.4
Decryption failure analysis	Part 3.5
LLL attack	Part 6
Size comparison	Part 7.2
Security discussion	Part 1, 2, 7

6.9 Suggested Reading Order

1. Part 1
2. Part 2
3. Part 3
4. Part 4
5. Part 5
6. Part 6
7. Part 7

6.10 Key Equations to Memorize

1. LWE relation: $\mathbf{b} = \mathbf{A}\mathbf{s} + \mathbf{e} \mod q$
2. Ring-LWE relation: $b = a \cdot s + e \in R_q$
3. Decryption: $d = v - s \cdot u \mod q$
4. Noise threshold: Decryption succeeds if noise $< q/4$
5. NTT: $\hat{a}_i = \sum_{j=0}^{n-1} a_j \omega^{ij} \mod q$
6. LLL size reduction: $|\mu_{i,j}| \leq 1/2$
7. LLL Lovász condition: $\|\mathbf{b}_i^*\|^2 \geq (\delta - \mu_{i,i-1}^2)\|\mathbf{b}_{i-1}^*\|^2$

6.11 What to Do Next

1. Read this guide once fully.
2. Re-read Parts 3, 4, and 5 carefully.
3. Work through a tiny example by hand: $n = 4$, $q = 17$.
4. Then start coding the polynomial arithmetic layer.

7 Work Distribution Among 3 Members

Below is a fair, balanced distribution for **3 members over 1 month**. Each person owns one technical pillar, but everyone must still understand the whole system for the viva.

7.1 Role Assignment Overview

Area	Member A (Ring Arithmetic & NTT Lead)	Member B (LWE Protocol & Encryption Lead)	Member C (Security & Comparison Lead)
Theory to master	Part 2, Part 4.2, Part 5	Part 3, Part 4, Part 3.5	Part 1, Part 2, Part 6, Part 7
Code files owned	`poly.py`, `ntt.py`	`sampler.py`, `rlwe.py`	`lll.py`, `experiments.py`
Main deliverables	Naive & NTT multiplication; NTT speedup benchmark	Keygen, encrypt, decrypt; correctness tests; noise sweep	From-scratch LLL; attack scaling; RSA/ECC size table
Report sections	Ring arithmetic, NTT, implementation details	LWE/Ring-LWE, encryption scheme, noise analysis	Quantum threat, LLL attack, comparison, security discussion
Viva focus	Explain $X^n + 1$, NTT, primitive root, why $q \equiv 1 \bmod 2n$	Explain $b = a \cdot s + e$, decryption equation, $q/4$ threshold	Explain LLL, lattice embedding, why RSA/ECC break under Shor

7.2 Weekly Work Distribution

7.2.1 Week 1 — Foundations & Naive Implementation

Day	Member A	Member B	Member C
1–2	Read Part 2, 4.2, 5.1–5.3. Hand-work $n = 4, q = 17$ polynomial multiplication.	Read Part 3, 4.1–4.4. Hand-work LWE encryption/decryption with tiny numbers.	Read Part 1, 2, 6.1–6.3. Implement LLL on a 2×2 matrix by hand.
3–4	Implement `poly_add`, `poly_sub`, `poly_mul_naive`, `poly_mod_reduction` in pure Python.	Implement discrete Gaussian sampler. Set up keygen/encrypt/decrypt using A's naive multiplication.	Implement LLL from scratch for small integer matrices. Test on 3×3.
5–6	Write tests comparing naive multiplication against hand example. Commit.	Write 100-trial encrypt/decrypt test. Verify correctness for bits 0 and 1.	Study LWE attack lattice. Set up RSA-2048/4096 and ECC-256 key generation using `cryptography` library for size baseline.
7	**Handoff:** A delivers `poly_mul_naive`. B integrates it. C verifies LLL works.		

Week 1 deliverable: Slow but correct Ring-LWE encryption/decryption + working LLL on tiny matrices.

7.2.2 Week 2 — NTT & Attack Implementation

Day	Member A	Member B	Member C
8–9	Learn NTT. Find primitive $2n$-th root of unity mod q. Implement forward NTT.	Integrate NTT into keygen/encrypt/decrypt once A delivers. Start parameter sweep for failure rate.	Construct LWE attack lattice. Run LLL for $n = 8, 12, 16$. Record attack time.
10–11	Implement inverse NTT. Test NTT multiplication against naive on random polynomials.	Verify NTT version produces identical ciphertexts to naive. Run 1000 encrypt/decrypt cycles; measure failure rate.	Extend attack to $n = 20, 24$. Document where LLL fails or times out.
12–13	Benchmark naive vs NTT for $n = 64, 128, 256$. Produce speedup plot.	Sweep noise $\sigma = 1, 2, 3.2, 5$. Plot failure rate vs σ. Mark $q/4$ threshold.	Compile size comparison table: Ring-LWE vs RSA-2048/4096 vs ECC-256.
14	**Handoff:** A delivers NTT. B integrates and confirms correctness. C delivers LLL attack results.		

Week 2 deliverable: Fast Ring-LWE with NTT + LLL attack for small n + noise data.

7.2.3 Week 3 — Experiments & Data Collection

Day	Member A	Member B	Member C
15–16	Optimize NTT. Produce final NTT speedup graph. Write NTT section of report.	Complete noise sweep. Produce final noise threshold graph. Write encryption/noise section.	Complete attack scaling plot (time vs n, exponential fit). Write LLL/attack section.
17–18	Compare Ring-LWE key size vs RSA/ECC using theoretical formulas.	Verify failure rate matches theoretical bound.	Build final size comparison table. Write comparison section.
19–20	Cross-review B and C code. Run integration tests.	Cross-review A and C code. Run integration tests.	Cross-review A and B code. Run integration tests.
21	**Checkpoint:** all plots and CSV data generated.		

Week 3 deliverable: All experimental plots, tables, and raw data.

7.2.4 Week 4 — Integration, Report, Viva Prep

Day	All Members
22–23	Write Jupyter notebook as lab report: Introduction → Math → Implementation → Experiments → Results → Discussion. Each writes own sections.
24–25	Write standalone 8–10 page report. A: arithmetic/NTT. B: protocol/noise. C: attack/comparison.
26–27	Prepare live demo: keygen → encrypt → decrypt → increase noise → LLL attack on tiny instance.
28–29	Rehearse viva. Each must explain all parts, but own sections deeply. Practice live code modification.
30	Final submission and presentation.

7.3 Cross-Training Schedule (Critical)

To ensure everyone learns, hold three 1-hour internal seminars:

Day	Teacher	Topic	Audience
3	Member A	Ring arithmetic and $X^n + 1$	B, C
4	Member B	LWE relation $b = a \cdot s + e$	A, C
6	Member C	LLL and lattice attacks	A, B
10	Member A	NTT and why it works	B, C
12	Member B	Noise threshold and $q/4$	A, C
15	Member C	RSA/ECC vs Ring-LWE trade-offs	A, B

7.4 Handoff & Dependency Schedule

Day	Deliverable	Owner	Consumer
4	`poly_mul_naive`	A	B
7	Working encrypt/decrypt (naive)	B	C
7	LLL on 3×3	C	A
10	NTT forward	A	B
12	NTT integrated	B	C
14	LLL attack for $n \leq 20$	C	A
16	Noise threshold plot	B	All
18	Size comparison table	C	All
20	Integration tests pass	All	All
25	Report draft	All	All
30	Final submission	All	Professor

7.5 Shared Responsibilities

- **GitHub repo:** Each member uses own branch, makes meaningful commits, and reviews at least one other member's pull request.
- **Lab notebook:** Each keeps dated notes with hand calculations, failed attempts, and parameter choices.
- **Weekly meetings:** 2× per week, 30 minutes. Each gives a 5-minute progress update.

- **Viva:** All three must be able to run the full notebook and explain any line of code. Each owns their section but must understand the rest.

7.6 Fairness Check

Member	Theory Load	Coding Load	Experiment Load	Writing Load
A	High (NTT)	High (poly, NTT)	Medium (speed)	Medium
B	High (LWE)	High (protocol)	High (noise)	Medium
C	High (LLL)	Medium (LLL)	High (attack, size)	High (comparison)

This is balanced. If one member is weaker, pair them with a stronger member for integration week.

7.7 Final Product Ownership

Final Artifact	Primary Owner	Secondary Owner
`poly.py`, `ntt.py`	A	B
`sampler.py`, `rlwe.py`	B	A
`lll.py`, `experiments.py`	C	B
Jupyter notebook	All	All
Report	All	All
Live demo	All	All

This distribution ensures each member has a clear, non-overlapping core, but the final product is genuinely collaborative and defensible in a viva.